

Applied 3 Conceptual Modelling

Applied 3 Conceptual Modelling

After completing this activity, you should be able to:

- develop a full conceptual model (on paper or using draw.io) from a supplied scenario
- use crows foot notation on a conceptual model according to the unit standards:
 - correct identification of the KEY attribute/s of an entity
 - correctly showing min and max cardinality on all relationships at both ends
 - correct naming via a verb of each relationship
 - correct indication of identifying vs non-identifying relationships.

This activity supports unit learning outcomes 1, 2 and 3.

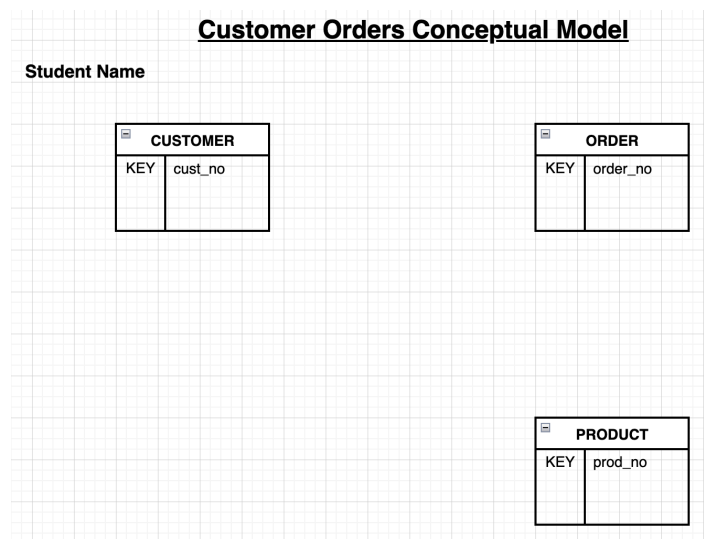
A3-1 Drawing an ER Diagram Using Draw.io

Given a scenario represented by the following entities, where customers place orders for products:

- CUSTOMER - customer number, name, address, phone number
- ORDER - order number, order date, total order value and for each product ordered, the quantity ordered and the total line price
- PRODUCT - product number, product description and product unit price

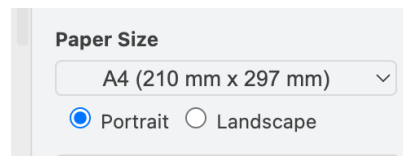
Step 1: Add entities and key attributes.

You should first draw an ERD that shows **entities and key attributes only**, as a preliminary analysis of the scenario you are modelling. An initial ERD using draw.io for the customer-orders scenario would be:



Prepare the ERD shown above using draw.io.

In the "Diagram" right panel, verify that the Paper Size is set to A4 and the orientation to Portrait.



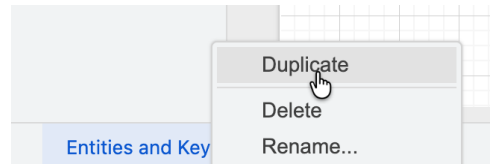
Use the "Misc" library Title item to add a title to your model and your student name. Ensure the model is saved in the appropriate Applied folder in your local repo, in a file named **cust_orders.drawio**. Add, commit and push the file to the FIT GitLab server.

Step 2: Add relationships.

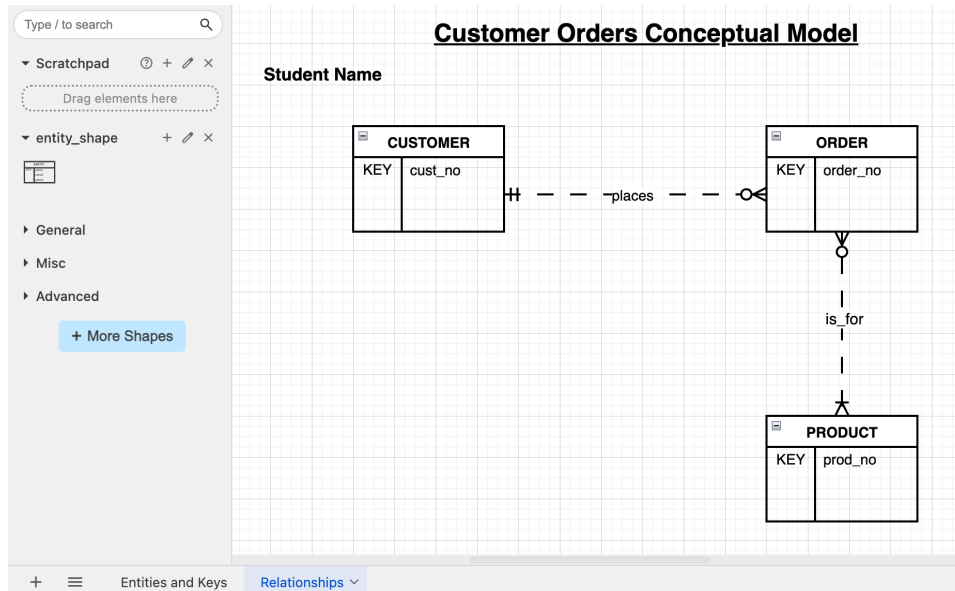


*Where possible M:N relationships should be left on a conceptual model as they make the model more straightforward and easier to visualise. We **only insert a bridging entity if required to represent an attribute which belongs to a relationship**.*

Rather than overwriting your current diagram, whenever you are about to make major changes, you should duplicate the current diagram into a new tab and then change this new tab. Note here that the original tab has been renamed to Entities and Keys, so its purpose is clear.



Our model with relationships added will have the form:



Save the model, then add, commit and push the file to the FIT GitLab server.

Note, as we have learnt, the way in which Git manages files (creates versions) means you do not, and **MUST NOT**, give models names such as cust_orders.drawio, cust_orders2.drawio etc. Simply use the same name all the time, i.e., cust_orders.drawio and overwrite the file in your local repo as you proceed. Git will manage the version control, as you saw in your last applied lesson.

Step 3: Add non-key attributes.

Duplicate your Relationships tab to a new tab called "final Model". During this step, it may be necessary to add attributes which need to be recorded in a relationship.

In the customer orders model above, if we wish to record the quantity of a product ordered and the total line price:

- These attributes cannot be placed in the ORDER entity (an order has potentially many products), nor
- Can they be placed in the PRODUCT entity (a product has potentially many orders)?

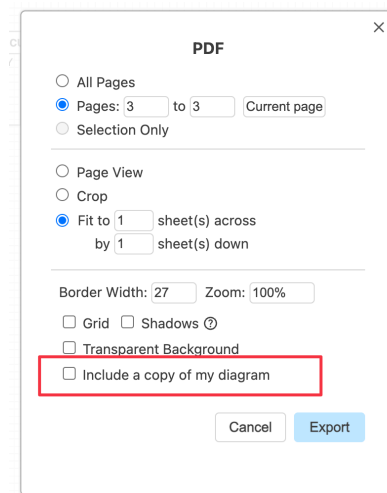
Here we need to add a new entity, ORDERLINE, which links one order to one product; this new entity converts the M:N relationship between ORDER and PRODUCT into two 1:M relationships. In naming this new entity, which is called an "associative entity" (or composite entity or bridging entity), when a natural name is not clear, you can use the parent entity names - here this would be

ORDER_PRODUCT.

As you work on your model, remember to regularly save it to your Applied folder in your local repo, then add, commit, and push the file to the FIT GitLab server. In this way, you provide a development history and a mechanism to "back track" to an older model if necessary.

Final Step.

When you have completed your model and made your last save, select File - Export As - PDF to create a PDF version of your final design in your local repo folder (output only the final model to PDF). In the PDF output options, ensure "Include a copy of my diagram" is unchecked:



Add, commit, and push this PDF you have generated to GitLab. In creating the PDF, please ensure it is fitted to 1 sheet across and 1 sheet down. In our unit, we will use your models' PDF output for grading by markers.

Possible Issues:

1. Underscore characters do not show or are faint

For some students, using underscores on draw.io - for example, cust_no, results in the underscores not appearing or being very faint. This is an issue with how your selected font is displayed. The solution is to use a different font for your model.

The suggested font to try is Verdana (at a size of 13 or 14):

- select All on your model (Edit menu - Select All)
- select the Text tab in the right-hand format panel and change the Font to Verdana

2. Pushing PDFs to GitLab results in an error

A push may return this error:

```
Git: RPC failed; HTTP 500 curl 22
The requested URL returned
error: 500
```

This error occurs due to file size limits when uploading documents you have created over HTTP.

If you receive this error, please edit your .config file in the .git folder and add the two lines below in bold after your current user details:

[user]

name =

email =

[http]

postBuffer = 524288000

A3-2 Applied Task 3 (AT3) - Assignment 1 Pre-Submission

Case Scenario

Prepare an Entity Relationship Diagram (ERD), showing all attributes and the identifier of each entity (KEY) for the following description of a Property Rental System:

- Properties are rented by tenants. Each rental recorded is by a particular tenant for a particular property. The only tenant details which are to be recorded in this system are those of the tenant arranging the rental. Each tenant is assigned a unique number by the Agency. Data held about tenants includes family name, given name, property rented, contact address - street, city, state, postcode & telephone number. A tenant may rent more than one property.
- Properties are owned by owners. Each property is assigned a unique property number. To deal with properties such as large shopping centres, each sub-property is treated as a property in its own right and assigned a property number. The agency only recognises a single owner for any of the properties it handles. The owner, address, and value are recorded for each property. Also, the lease period (e.g., 24 months) and bond are recorded for each property or sub-property rented. An owner may own several properties. For each owner, an owner number is assigned, and the owner's name is also recorded.
- Properties are subject to damage. The agency records all instances of damage to its properties caused by tenants, including the date, type of damage, and repair cost. Repair costs are charged directly to tenants, who must pay for each damage in a single payment.
- Tenants pay accounts to the agency - these consist of weekly rental payments, bond payments (for new properties) and damage bills. The date of payment, tenant, property, type of account (Rental, Bond, Damage) and amount are recorded. Each payment is assigned a unique payment number.

The steps to complete this Applied Task 3:

Using draw.io, prepare a **FULL conceptual model** (Entity Relationship Diagram) using crow's foot notation for Property Rental System (PRS) as described above.

- For this FULL conceptual model (ERD), include:
 - Identifiers (**keys**) for each entity
 - all required attributes and
 - all relationships. Cardinality (min and max) and connectivity for all relationships must be shown on the diagram.
- **Surrogate keys must not be added to this model.**

Your model must conform to the unit ERD standards listed in *Topic 2 Core Reading and Quiz* lesson. Your name must be shown on your diagram, and it must be exported as an A4 portrait page.

Submission requirement (due: Monday, 23 March 2026, 9:00 AM):

The following two files **must be submitted to Moodle** and **must also exist** in your FITGitLab server repo:

1. A **single-page pdf file** representing your final conceptual model. Name the file **prs_conceptual.pdf**. This file must be created using File - Export As - PDF from draw.io (do not use screen capture) and must be accessible with a development history via Git (your

prs_conceptual.drawio file). Ensure you select Fit to 1 sheet(s) across and 1 sheet(s) down in the export pop-up screen.

2. A PDF document containing any assumptions you wish to make your marker aware of, or further information that you may require from the client. Name the file **prs_assumptions.pdf**. If you have made no assumptions or required information, submit the document with a single statement saying, "No assumptions made and no further information needed".

Marking Criteria

This assessed Applied task contributes to your Assignment 1 grade and carries a maximum of 1 mark. Submitting the task and achieving a pass grade (≥ 0.5) will provide early feedback on your conceptual modelling skills and allow access to the Assignment 1 brief and submission link, when available. If you receive a fail, you can resubmit until you pass.

Note that your marker needs at least five business days to grade this task. You must submit the task by the due date to get on-time feedback and access to Assignment 1. Any late submission will impact your access to the Assignment 1 brief and ultimately affect your Assignment 1 progress. Submissions more than 7 days late will not be accepted.